ECE 583 – FPGA Design (Fall 2026)

Instructor: Professor Dean Sullivan

ALU Take Home

Student: Yago Martínez de Fuenmayor

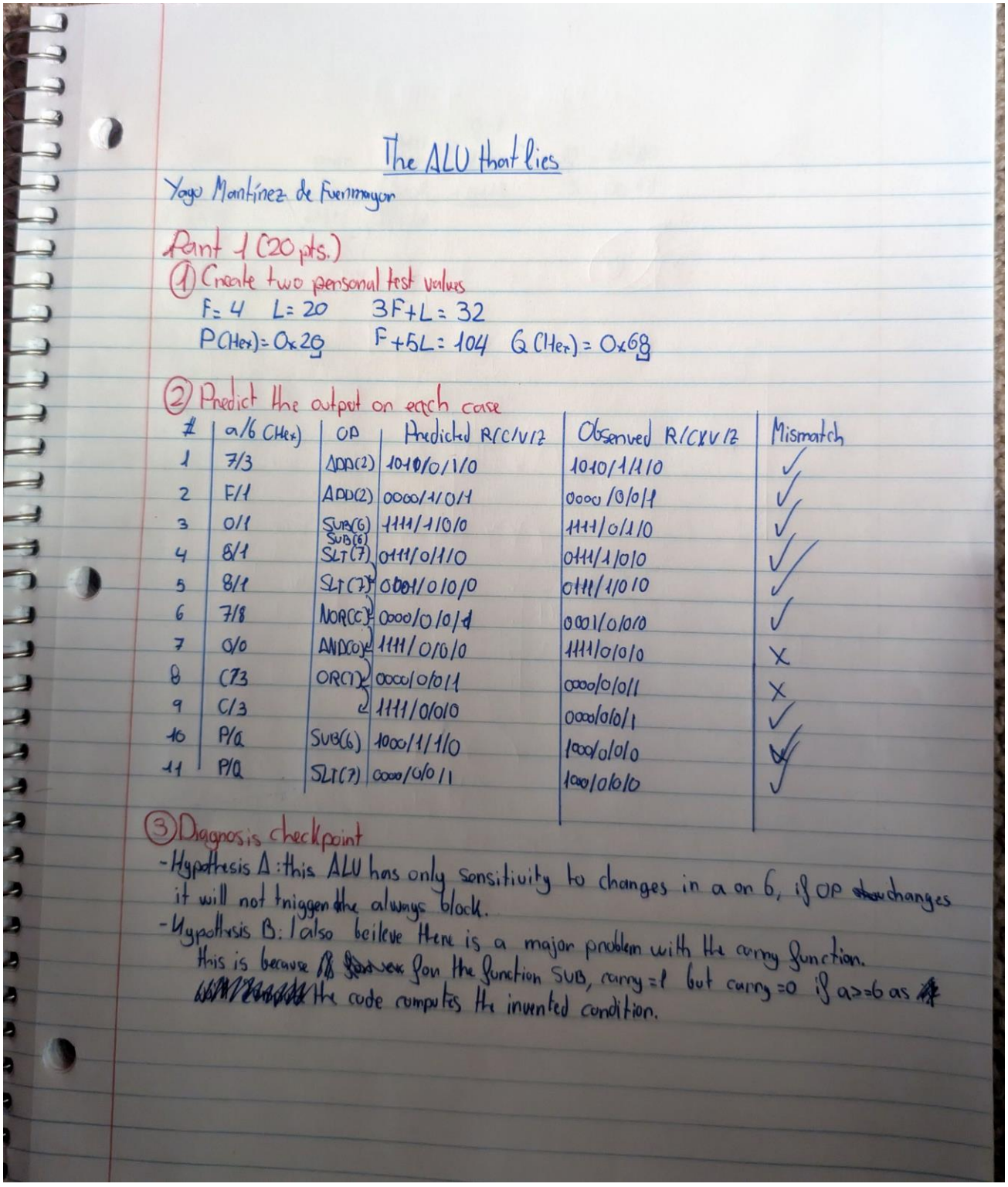
Lab Section: 01

Department of Electrical and Computer Engineering

University of New Hampshire

Submission Date: 10/6/2026

1. Part 1



The ALU that lies

Yago Martínez de Fuentemayor

Part 1 (20 pts.)

① Create two personal test values

F = 4 L = 20 3F+L = 32

P (Hex) = 0x20 F+5L = 104 G (Hex) = 0x68

② Predict the output on each case

#	a/b (Hex)	OP	Predicted R/C/V/Z	Observed R/C/V/Z	Mismatch
1	7/3	ADD(2)	1010/0/1/0	1010/1/1/0	✓
2	F/1	ADD(2)	0000/1/0/1	0000/0/0/1	✓
3	0/1	SUB(6)	1111/1/0/0	1111/0/1/0	✓
4	8/1	SUB(6) SLT(7)	0111/0/1/0	0111/1/0/0	✓
5	8/1	SLT(7)	0001/0/0/0	0111/1/0/0	✓
6	7/8	NOR(C)	0000/0/0/1	0001/0/0/0	✓
7	0/0	AND(0)	1111/0/0/0	1111/0/0/0	X
8	C/3	OR(1)	0000/0/0/1	0000/0/0/1	X
9	C/3		1111/0/0/0	0000/0/0/1	✓
10	P/Q	SUB(6)	1000/1/1/0	1000/0/0/0	✓
11	P/Q	SLT(7)	0000/0/0/1	1000/0/0/0	✓

③ Diagnosis checkpoint

- Hypothesis A: this ALU has only sensitivity to changes in a or b, if OP changes it will not trigger the always block.
- Hypothesis B: I also believe there is a major problem with the carry function. This is because for the function SUB, carry = 1 but carry = 0 if a >= b as the code computes the inverted condition.

Image 1. Hand written part 1

1.1. Simulation.

1.1.1. Test Bench.

```
`timescale 1ns / 1ps

module tb_decimal_bcd_encoder;

    reg i1, i2, i3, i4, i5, i6, i7, i8, i9;
    wire d, c, b, a;

    decimal_bcd_encoderDF uut (
        .i1(i1),
        .i2(i2),
        .i3(i3),
        .i4(i4),
        .i5(i5),
        .i6(i6),
        .i7(i7),
        .i8(i8),
        .i9(i9),
        .d(d),
        .c(c),
        .b(b),
        .a(a)
    );

    initial begin
        i1 = 1; i2 = 1; i3 = 1; i4 = 1; i5 = 1;
        i6 = 1; i7 = 1; i8 = 1; i9 = 1;

        #50;
        i9 = 0;

        #50;
        i9 = 1; i8 = 0;

        #50;
        i8 = 1; i7 = 0;

        #50;
        i7 = 1; i6 = 0;

        #50;
        i6 = 1; i5 = 0;
```

```
        #50;
        i5 = 1; i4 = 0;

        #50;
        i4 = 1; i3 = 0;

        #50;
        i3 = 1; i2 = 0;

        #50;
        i2 = 1; i1 = 0;

        #50;
        i1 = 1;

        $finish;
    end

endmodule
```

Include 1. Test bench for the wrong ALU in Verilog.

1.1.2. Simulation.

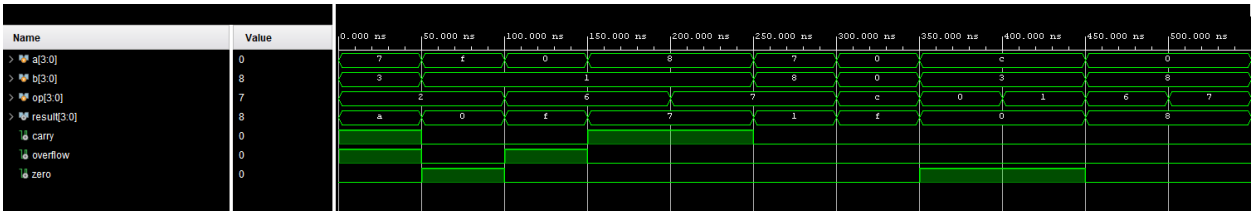


Image 1. Simulation graph of the wrong ALU generated by Vivado.

2. Part 2 – Fixed ALU
   2.1. Repaired ALU

```
`timescale 1ns / 1ps

module alu4_fixed (
   input wire [3:0] a, b,
   input wire [3:0] op,
   output reg [3:0] result,
   output reg carry, overflow, zero
   );

   reg [4:0] temp;

   always @(*) begin
      // non-arithmetic ops or unhandled opcodes
      result   = 4'b0000;
      temp     = 5'b00000;
      carry    = 1'b0;
      overflow = 1'b0;

      case (op)
         4'b0000: result = a & b;
         4'b0001: result = a | b;
         4'b0010: begin
            temp     = a + b;
            result   = temp[3:0];
            carry    = temp[4]; // Bit 4 represents carry-out past bit 3
            overflow = (a[3] == b[3]) && (result[3] != a[3]);
         end
         4'b0110: begin
            temp     = a - b;
            result   = temp[3:0];
            carry    = (a < b); // Carry = 1 if an unsigned borrow is required
            overflow = (a[3] != b[3]) && (result[3] != a[3]); // Correct SUB overflow
detection
         end
         4'b0111: begin
            // Signed comparison ($signed) for Set Less Than
            result   = ($signed(a) < $signed(b)) ? 4'b0001 : 4'b0000;
         end
         4'b1100: result = ~(a | b);

         // All unhandled opcodes default to result=0000, carry=0, overflow=0
```

```
            default: result = 4'b0000;
        endcase

        // Zero flag evaluated on the final 4-bit result for ALL opcodes
        zero = (result == 4'b0000);
    end

endmodule
```

Include 3. Behavioral implementation of a 74147 Decimal to BCD priority encoder in Verilog.

2.2. Independent oracle - Simulation.

2.2.1. Test bench.

```
`timescale 1ns / 1ps

module tb_alu4;

    reg [3:0] a;
    reg [3:0] b;
    reg [3:0] op;

    wire [3:0] result;
    wire carry;
    wire overflow;
    wire zero;

    integer total_tests = 0;
    integer mismatch_count = 0;

    reg [3:0] exp_R;
    reg exp_C;
    reg exp_V;
    reg exp_Z;
    reg [4:0] add_ext;

    alu4_fixed uut (
        .a(a),
        .b(b),
        .op(op),
        .result(result),
        .carry(carry),
        .overflow(overflow),
        .zero(zero)
```

```
);

integer total_tests = 0;
integer mismatch_count = 0;

reg [3:0] exp_R;
reg exp_C;
reg exp_V;
reg exp_Z;
reg [4:0] add_ext;

// Apply vectors, wait and report failures
task run_test;
   input [3:0] test_a;
   input [3:0] test_b;
   input [3:0] test_op;
   begin
      a  = test_a;
      b  = test_b;
      op = test_op;

      #50;

      exp_R = 4'b0000;
      exp_C = 1'b0;
      exp_V = 1'b0;

      case (op)
         4'b0000: exp_R = a & b; // AND
         4'b0001: exp_R = a | b; // OR
         4'b0010: begin // ADD
            add_ext = {1'b0, a} + {1'b0, b};
            exp_R   = add_ext[3:0];
            exp_C   = add_ext[4];
            exp_V   = (a[3] == b[3]) && (exp_R[3] != a[3]);
         end
         4'b0110: begin // SUB
            exp_R   = a - b;
            exp_C   = (a < b);
            exp_V   = (a[3] != b[3]) && (exp_R[3] != a[3]);
         end
         4'b0111: exp_R = ($signed(a) < $signed(b)) ? 4'b0001 : 4'b0000; // SLT
         4'b1100: exp_R = ~(a | b); // NOR
```

```
            default: exp_R = 4'b0000; // Invalid opcodes default to 0
        endcase

        exp_Z = (exp_R == 4'b0000);
        total_tests = total_tests + 1;

        // Use !== to ensure X or Z states cannot silently pass
        if ((result !== exp_R) || (carry !== exp_C) || (overflow !== exp_V) || (zero !==
      exp_Z)) begin
            $display("[%0t ns] FAIL: a=%h, b=%h, op=%b | OBSERVED: R=%h C=%b
      V=%b Z=%b | EXPECTED: R=%h C=%b V=%b Z=%b",
                     $time, a, b, op, result, carry, overflow, zero, exp_R, exp_C, exp_V,
      exp_Z);
            mismatch_count = mismatch_count + 1;
        end
    end
endtask

// Control variables for loops
integer op_idx, a_val, b_val;
reg [3:0] legal_ops [0:5];

initial begin

    exp_R  = 4'b0;
    exp_C  = 1'b0;
    exp_V  = 1'b0;
    exp_Z  = 1'b0;
    op_idx = 0;
    a_val  = 0;
    b_val  = 0;

    legal_ops[0] = 4'b0000; // AND
    legal_ops[1] = 4'b0001; // OR
    legal_ops[2] = 4'b0010; // ADD
    legal_ops[3] = 4'b0110; // SUB
    legal_ops[4] = 4'b0111; // SLT
    legal_ops[5] = 4'b1100; // NOR

    $display("Starting Comprehensive Self-Checking ALU Testbench");

    run_test(4'h7, 4'h3, 4'b0010); // ADD Positive Overflow
    run_test(4'hF, 4'h1, 4'b0010); // ADD Carry-out & Zero
```

```
run_test(4'h0, 4'h1, 4'b0110); // SUB Unsigned Borrow
run_test(4'h8, 4'h1, 4'b0110); // SUB Negative Overflow
run_test(4'h0, 4'h1, 4'b0111); // SLT True
run_test(4'h7, 4'h8, 4'b0111); // SLT False
run_test(4'h0, 4'h0, 4'b1100); // NOR
run_test(4'hC, 4'h3, 4'b0000); // AND Zero flag

// P, Q Tests (P=0, Q=8)
run_test(4'h0, 4'h8, 4'b0110); // SUB P(0), Q(8)
run_test(4'h0, 4'h8, 4'b0111); // SLT P(0), Q(8)

// All 16x16 combinations
for (op_idx = 0; op_idx < 6; op_idx = op_idx + 1) begin
   for (a_val = 0; a_val < 16; a_val = a_val + 1) begin
      for (b_val = 0; b_val < 16; b_val = b_val + 1) begin
         run_test(a_val[3:0], b_val[3:0], legal_ops[op_idx]);
      end
   end
end

// Unhandled Opcode Test (e.g., 4'b1111)
run_test(4'h5, 4'h3, 4'b1111);

a = 4'hC; b = 4'h3;
op = 4'b0000; #50 run_test(4'hC, 4'h3, 4'b0000);
op = 4'b0001; #50 run_test(4'hC, 4'h3, 4'b0001);
op = 4'b0010; #50 run_test(4'hC, 4'h3, 4'b0010);
op = 4'b0110; #50 run_test(4'hC, 4'h3, 4'b0110);
op = 4'b0111; #50 run_test(4'hC, 4'h3, 4'b0111);
op = 4'b1100; #50 run_test(4'hC, 4'h3, 4'b1100);

    #50;

// Summary Report
$display("Testbench Complete");
$display("Total Tests Executed : %0d", total_tests);
$display("Total Mismatches     : %0d", mismatch_count);
if (mismatch_count == 0) begin
   $display("STATUS: PASS");
end else begin
   $display("STATUS: FAIL");
end
```

```
        $finish;
    end

endmodule
```

Include 2. Test bench for the corrected ALU encoder in Verilog.

2.2.2. Simulation.

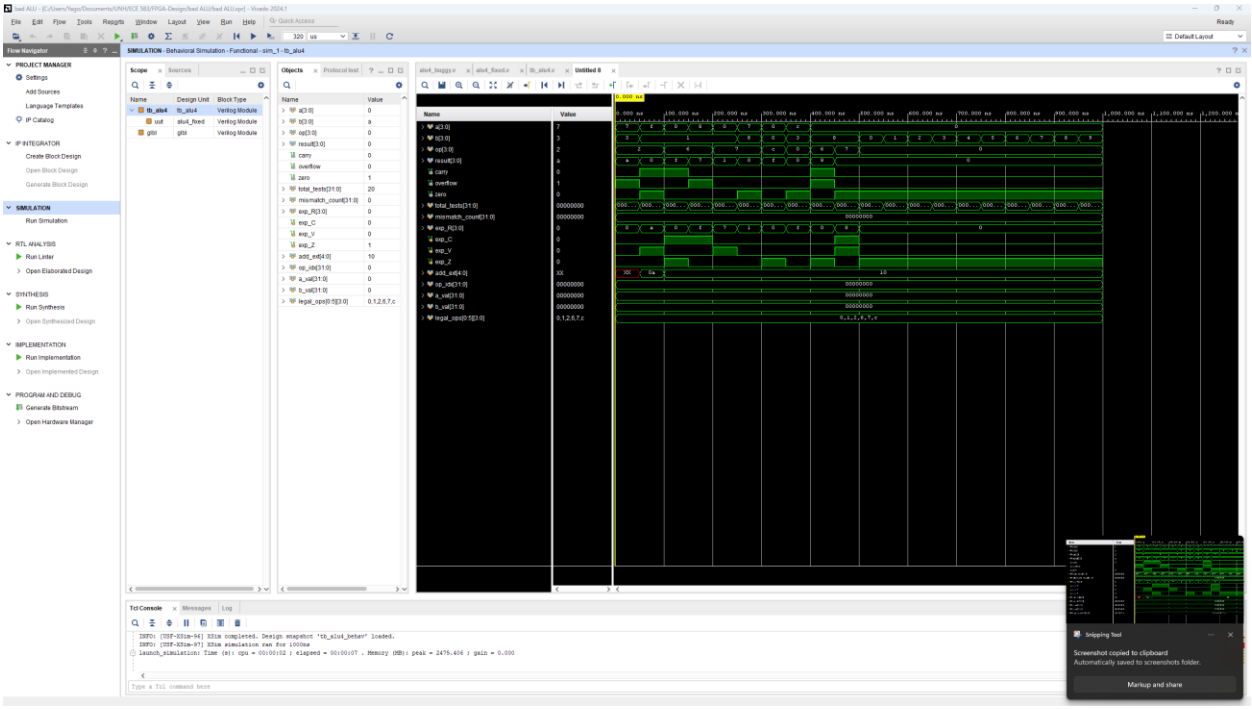


Image 4. Simulation graph of the complex simulation generated by Vivado.

3. AI disclosure.
   AI was used in several instances in the creation of this report. It was used to debug the complex testbench due to its complexity. Several times I faced problems with some outputs displaying errors. It was also used to generate pseudocode and explanations of different nuances and concepts of verilog for the testbench.
   AI was also used to proofread this report.